kerneltests: Verified and Fast Aggregated Kernel Hypothesis Tests in Python

Ankith Burki

Abstract

Aggregated kernel tests—MMDAgg for two-sample testing, KSDAgg for goodness of fit, and their incomplete U-statistic variants—are among the most powerful nonparametric tests available, adapting to unknown smoothness without data splitting. Their implementations are spread over separate single-paper repositories with different interfaces and no test suites. `kerneltests` unifies them behind one API, verifies every port against the original reference code to bit-exact agreement, fixes two confirmed defects in the references, and provides a faster algebraic evaluation of the wild bootstrap (1.7–2.3× on CPU). The library is MIT-licensed, has [TODO: N]% line coverage across [TODO: OS/Python matrix] in continuous integration, and is documented at [TODO: URL].

1 Introduction

Kernel hypothesis tests compare distributions through reproducing kernel Hilbert space embeddings: the maximum mean discrepancy (MMD) for two-sample testing, the Hilbert–Schmidt independence criterion (HSIC) for independence, and the kernel Stein discrepancy (KSD) for goodness of fit. Their power depends strongly on the kernel bandwidth. Aggregated tests (Schrab et al., 2023, 2022a) combine a collection of bandwidths and kernels with a level correction, achieving minimax-optimal power over Sobolev balls without data splitting, and incomplete U-statistics (Schrab et al., 2022b) reduce their cost from quadratic to linear in the sample size.

These methods are used in [TODO: applications with citations: distribution-shift detection, generative-model evaluation, simulation-based inference checks]. Their reference implementations are research code: one repository per paper, no automated tests, asserts in place of input validation, and inconsistent interfaces. `kerneltests` provides a single maintained library aimed at practitioners and at researchers who need trustworthy baselines.

2 Design

The public API has one function per testing problem, each returning a `TestResult` with the decision, the level, and per-kernel diagnostics (statistics, quantiles, p-values, thresholds):

```
import kt
res = kt.two_sample(X, Y)                # MMDAgg
res = kt.goodness_of_fit(X, score_X)     # KSDAgg
res = kt.independence(X, Y)              # HSICAggInc   (TODO: lands with agginc port)
```

Low-level functions expose every option of the original methods (19 kernel configurations, four weighting schemes, wild bootstrap or permutations, parametric bootstrap for KSD). Weights, bandwidth selection and kernels are shared across tests. The only dependencies are NumPy and SciPy.

3 Verification

Correctness of a statistical test has two parts: agreement with the published method, and control of the type-I error. For the first, each port reproduces the reference random-number protocol, so for a given seed the decision, statistics, quantiles and thresholds can be compared exactly. MMDAgg agrees in all 152 configurations tested (every kernel option × weighting × odd/even bandwidth counts, both resampling schemes) and KSDAgg in all 30 configurations tested. For the second, rejection rates under the null are checked by simulation in the test suite [TODO: report a proper level study: several dimensions, sample sizes, ≥ 1000 repetitions].

Exact comparison exposed two defects in the reference implementations, both reported upstream [TODO: issue links]: (i) with the default `laplace_gaussian` kernel, MMDAgg reports the Laplace-kernel p-values in place of the Gaussian ones (an indexing error; decisions unaffected); (ii) KSDAgg compares two equivalent rejection rules with an exact-equality assertion, so floating-point rounding makes it raise an error on some ordinary inputs. `kerneltests` uses a single rejection rule and includes regression tests for both.

4 Performance

For equal sample sizes the wild-bootstrap statistic is $r^\top K r$ with $r = (\epsilon, -\epsilon)$ and $K \in \mathbb{R}^{2n \times 2n}$; this equals $\epsilon^\top H \epsilon$ with $H = K_{XX} + K_{YY} - K_{XY} - K_{YX} \in \mathbb{R}^{n \times n}$, a fourfold reduction in matrix-product cost.

n	reference (s)	kerneltests (s)	speedup
500	1.61	0.95	1.7×
1000	3.15	1.38	2.3×
2000	14.48	8.28	1.7×

Single run, $d = 10$, $B_1 = B_2 = 500$, one CPU machine [TODO: repeat with medians over runs, report hardware, add memory, add the incomplete U-statistic tests and a GPU backend].

5 Comparison with existing software

[TODO: feature/runtime table vs hyppo, the per-paper packages (mmdagg, ksdagg, agginc, mmdfuse), torch-two-sample, PyRKHSstats; verify each row].

6 Community and availability

Source: [TODO: GitHub URL]; documentation: [TODO: URL]; PyPI: [TODO: package]; issue tracker and contribution guide are public. [TODO: user base evidence: downloads, dependents, external contributors, issues from users].

Acknowledgments

[TODO: e.g. thanks to the authors of the reference implementations]

References

A. Schrab, I. Kim, M. Albert, B. Laurent, B. Guedj, A. Gretton. MMD aggregated two-sample test. *JMLR* 24(194):1–81, 2023.

A. Schrab, B. Guedj, A. Gretton. KSD aggregated goodness-of-fit test. *NeurIPS*, 2022.

A. Schrab, I. Kim, B. Guedj, A. Gretton. Efficient aggregated kernel tests using incomplete U-statistics. *NeurIPS*, 2022.